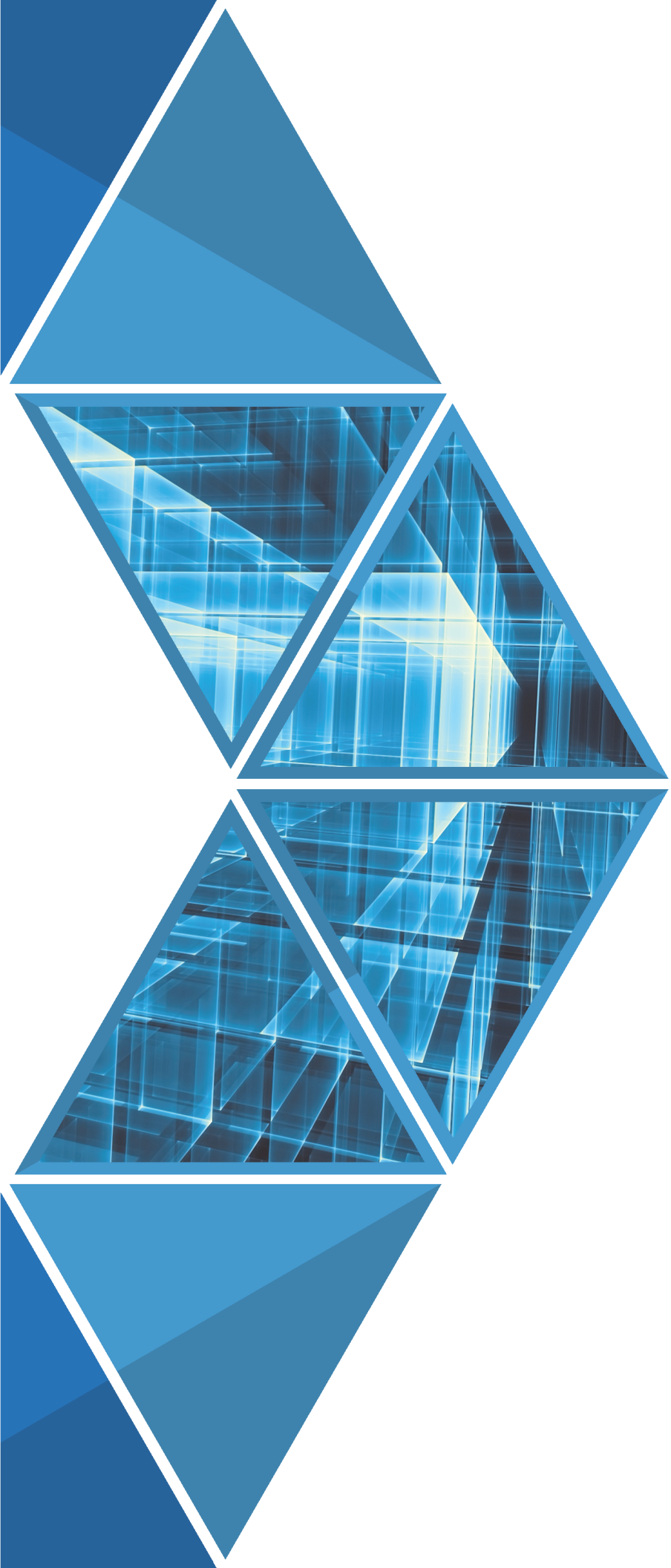


MemVerge

Memory Machine™

System Resiliency Guide

Beta Release 1

Published: 2021-08-24

Last Revised: 2021-10-05

Copyright 2021 - MemVerge. All rights reserved.

Contents

Introduction **3**

Architecture Overview **4**

The Data Service 4

The Memory Machine Management Service 5

Applications, Instances and Snapshots 6

Ancillary Systems 7

Memory Machine Failure Modes **9**

Memory Machine Data Service Failures 9

Management Service Failures 10

Hardware Failure Modes 11

Introduction

MemVerge Memory Machine introduces a new paradigm that adds enormous value to memory-intensive computation by managing volatile and persistent memory and by providing an array of services made possible by nearly instant in-memory snapshots.

To achieve these benefits, MemVerge Memory Machine necessarily adds a layer of complexity to runtime memory management software. This guide describes failure modes that are unique to this layer. The guide is intended to aid in technical evaluation and due diligence with respect to adding the MemVerge solution to critical software application stacks.



Note: This is not a troubleshooting guide. This document describes causes of failure modes for purposes of evaluating reliability and fitness of purpose. Descriptions of failure effects in this document are incomplete and in practice can require significant root cause analysis to distinguish.

The guide is organized as follows:

- Architecture Overview *(on page 4)* briefly describes the architecture of MemVerge Memory Machine. For a more detailed explanation of memory management and MemVerge's architecture, see the *Memory Machine™ 2.0: Internal Architecture*.
- Memory Machine Data Service Failures *(on page 9)* describes failure modes and likely consequences of data service components.
- Management Service Failures *(on page 10)* describes failure modes and likely consequences of management service components.
- Hardware Failure Modes *(on page 11)* describes hardware-related failures and how they affect Memory Machine.

Architecture Overview

This chapter is a brief overview of the components that make up the Memory Machine system.

Memory Machine consists of two core systems:

- A *Data Service* that manages application data in persistent memory (PMEM) and manages data tiered between PMEM and volatile memory (DRAM) to optimize application performance.
- A *Management Service* that performs non-data tasks such as messaging between hosts; running the user interface; and gathering metrics.

Memory Machine also includes a cluster snapshot service and a cluster Management Service. These components are out of the scope of this document. For information about these services and for a more complete description of the Memory Machine architecture, see the *MemVerge Memory Machine User's Guide*.

The Data Service

Memory Machine's Data Service:

- Manages tiered memory by assigning physical DRAM and PMEM to virtual addresses
- Allocates and deallocates PMEM for any number of application instances
- Optimizes application performance by tiering in-demand memory pages from PMEM to DRAM

The data service consists of two components, the Transparent Memory Service (TMS) and the MemVerge Memory Allocation Service (MVMAS). To understand these components, it is first necessary to understand memory tiering.

Memory Tiering

Caching is the copying of "hot" data (data that is read or written frequently and/or recently) to an area of faster working memory closer to the processor. The data remains in "main" memory while it is cached. When the cache fills up, old data is simply removed from cache and replaced with hotter data. When the old data is accessed, the cache "misses" and the data is read from its original location.

Tiering, the technique used by Memory Machine, is similar but distinct from caching. In tiering, hot data is moved to a faster tier of memory (DRAM), but the data is then removed from the slower tier, freeing it for use. Similarly, when tiered data is evicted from DRAM (to make room for hotter data), it is written back to the slower data tier (PMEM). There is no concept of a cache "hit" or "miss" in tiering; rather, a virtual address is mapped to the physical location of the data, and only one copy of the data exists at a time.

Memory Machine tracks usage statistics of data in memory and manages tiering between PMEM and DRAM to optimize performance. Memory Machine maintains a user-configurable pool of DRAM tier memory for every application instance and intelligently moves hot data into this pool, evicting colder data back to PMEM if necessary.

Transparent Memory Service

Most applications on Linux are dynamically linked to the GNU C core library `glibc.so`, which provides memory allocation functions such as `mmap()` and `malloc()`. Linux offers the ability to intercept and reroute calls to these functions by naming a preloaded library with identical function names. The file path of this preloaded library is specified in the `LD_PRELOAD` environment variable. The preloaded library (if it has been specified) shadows the `glibc.so` library and intercepts its function calls.

MemVerge provides a shared library, Tiered Memory Service (TMS), that overloads `glibc.so`'s memory allocation functions.

Memory Machine TMS intercepts Linux `glibc.so` memory allocation function calls from an application. The function call returns a virtual address to the application. Behind the scenes, TMS tracks where the data is actually stored in memory – PMEM or DRAM tier – and manages tiering between PMEM and DRAM.

When PMEM is allocated to fulfill a memory allocation request, TMS calls MemVerge Memory Allocation Service (MVMAS), a service that maps byte-addressable memory to the Memory Machine PMEM store.

Memory under the management of Memory Machine is referred to in MemVerge documentation as being *under management* or as *managed memory*. These terms refer to both PMEM that is under MVMAS control and DRAM memory allocated for use as tier memory.

MemVerge Memory Allocation Service

The Memory Machine MVMAS is a daemon, `mvmallocd`, that runs on every Memory Machine data host. The daemon is managed by the host's service configuration.

MVMAS is at the heart of Memory Machine. MVMAS manages memory allocation to persistent memory (PMEM) for all applications run in Memory Machine by mapping programmatic memory calls to the Memory Machine PMEM store.

The Memory Machine Management Service

The Memory Machine Management Service:

- Runs the web server for *Memory Machine Management Center* (*M3C*)
- Services API requests
- Manages and stores configurations
- Gathers metrics
- Generates logs and reports
- Tracks and records license compliance
- Communicates with MemVerge if Call Home is active
- Manages scheduled jobs such as scheduled snapshots

The Management Service consists of two components: a Control Path Service (CPS) running on a management host and Memory Machine Agents on all data hosts.

Control Path Service

CPS is a service that manages aspects of Memory Machine not having to do with data.

Unlike Data Service, CPS runs on only a single designated management host. CPS is a daemon named `mmctl` that is managed by the host's service configuration. CPS can run concurrently withData Service on a data host or can run on a dedicated host or VM.

Memory Machine Agent

CPS communicates with a Memory Machine Agent daemon, `mmagent`, running as a service on each data host. The agent serves as an intermediary between CPS and the Data Service on the host.

The following information is passed from the Memory Machine Agent to CPS:

- A "heartbeat" that periodically confirms that the host data service is running.
- The configuration and status of each application process on the host.
- PMEM and DRAM usage statistics for each application process on the host.
- System information about the host.
- Messages logged by the data service.

Applications, Instances and Snapshots

An *application* is a software program for which Memory Machine manages volatile and persistent memory. Multiple copies of the same application are often run on a single host as separate processes. Each process has a unique process ID (PID) assigned by the operating system. (In the case of multi-process or multithreaded applications, the process is represented by the PID of the root process.)

Each process is assigned a configuration, which is read from a configuration file called `mvmalloc.yml` whose default directory path is `/etc/memverge/`. The configuration specifies which memory allocation commands to intercept and how much DRAM to assign to tier memory, among other things. Processes can have different configurations, even for the same application on the same host.

A *snapshot* is a "frozen" application process captured in-place in PMEM. Any part of the process that was running in DRAM tier is flushed to PMEM, and some metadata and an identifier are written to a store on a storage device. DRAM not under Memory Machine management is flushed to storage.

A snapshot can can be restored so that the process resumes running from the exact state at which it was captured. A process can have up to 256 snapshots representing its state at various points in its run.

An *application instance* consists of one or more snapshots of the same process, with the same PID and runtime configuration. It might or might not also contain a running version of the process. An application instance containing no snapshots consists solely of the running process (or process tree).

Normally you use a supplied "wrapper" script, `mm`, to run an application in Memory Machine. This script initializes the runtime environment using settings in the MVMAS and application configuration files.

Ancillary Systems

All other components required for Memory Machine to run are described below.

Licensing

Memory Machine requires a software license in order to manage persistent and tiered DRAM memory. Memory Machine usage is licensed based on the total amount of memory under management. Three types of license are available:

- On-Demand License: A multi-host "pay for what you use" license for Memory Machine Advanced Edition. A license server polls all managed hosts and aggregates memory statistics to determine billable usage.
- Reserved License: A multi-host license for Memory Machine Advanced Edition with a set memory limit. A license server polls all involved hosts and aggregates memory usage statistics to verify license compliance.
- Standard License: A license for Memory Machine Standard Edition with memory management but no data services such as snapshots. A standard license can be multi-host or can be a *file-based* license bound to a data host.

Multi-host licenses are managed in the *Memory Machine Management Center* (*M3C*).

Memory Machine Management Console

The *Memory Machine Management Center* (*M3C*) is a web-based UI for configuring, managing, and monitoring Memory Machine hosts. The *M3C* server is served by the `mmctl` daemon of the Management Service.

Database

Memory Machine uses a database management system (DBMS) to store data and control system parameters, metrics, and configurations.

This database is a potential critical point of failure, especially for the Management Service.

Memory Machine offers integration with a growing number of databases that provide varying levels of failover security.

Open Source Software

Memory Machine uses a number of open source software (OSS) libraries and APIs. For a list of OSS software used by Memory Machine, see the *Memory Machine Open Source Licenses* document. If you have questions about OSS failure modes could affect Memory Machine, contact MemVerge customer support.

Alerts and Logs

The Management Service raises alerts for important events such as running out of memory, approaching license expiration, and others. Alerts generate messages in *M3C* and can optionally generate email notifications.

The Management Service also generates usage reports. The management server can optionally be configured to send usage reports to MemVerge (telemetry) for billing and troubleshooting. By default telemetry is disabled.

All Memory Machine components write event logs. Log levels are configurable for most components. Memory Machine log files are:

- `malloc_log_<appname_pid_timestamp>`: the MemVerge TMS log. There is a TMS log for every managed process; log files are named using the process ID (PID) and time that the process started.
- `mmagent.log`: the log for the Memory Machine Agent process
- `mmctl.log`: the log for the Memory Machine Management Service
- `mvmallocd_log`: the MVMAS information log
- `mvsnapd_log`: the log for the Memory Machine snapshot service
- `mvcsnapd_log`: the log for the Memory Machine cluster snapshot service

The `mmctl.log` is written only on the management host. All other logs are written on all data hosts.

Memory Machine Failure Modes

This section details failure modes that are unique to the Memory Machine memory management system.

Memory Machine Data Service Failures

This section describes failure modes of the Memory Machine data components.

TMS Failure

TMS is a library, not a process. When in use, TMS is dynamically linked to an application and can be considered part of the application code for purposes of failure analysis. See Application Failure *(on page 9)* below.

Application Failure

In general, the effect of an application failure depends on the nature of the application. If an application does not corrupt shared resources, it should not affect other processes using managed memory.

A failed application can be restarted from a persisted snapshot if one exists.

If more than one snapshot have been captured, the application instance can be restored from any of them. Usually the most recent snapshot is selected, assuming that a later snapshot represents more completed work that would take time and resources to reproduce from an earlier snapshot. Scheduled snapshots can be configured to various strategies such as a rolling window.

MVMAS Failure

If MVMAS becomes unavailable, a TMS instance cannot acquire new allocated PMEM for its associated application. When this happens, the Management Service generates and logs a `Memory Service is Down` alert.

TMS is designed to fall back on DRAM operation in the absence of new PMEM allocation. If TMS cannot connect to MVMAS:

- TMS fulfills all new allocation requests with unmanaged DRAM. That is, it passes allocation requests through to the Linux standard library `glibc`.
- TMS continues to try to contact MVMAS. If TMS reconnects with MVMAS, it resumes using PMEM for new allocation requests. However, it cannot retroactively replace unmanaged memory. Any memory allocated while MVMAS was down remains unmanaged.
- While MVMAS is unavailable, TMS stops tiering between DRAM and PMEM.
- TMS continues to collect page hit and longevity metrics for tiering.

PMEM allocated before MVMAS failure is available as normal. PMEM can be read normally by the application (recall that the application sees only a logical memory address that happens to be backed with physical PMEM).

Snapshot activity halts if MVMAS is lost and resumes when MVMAS becomes available.

Under normal circumstances the host service configuration restarts MVMAS. When MVMAS is restored:

- PMEM allocation becomes available again.
- Memory that was allocated to DRAM while MVMAS was down remains in unmanaged DRAM.
- Snapshots that are not yet persisted can use PMEM to persist DRAM as it is flushed.

Management Service Failures

This section describes failure modes of the Memory Machine management components.

Control Path Service Failure

In general CPS cannot cause data failures, but it can limit the functionality of Memory Machine. For example:

- CPS can force a data host's MVMAS into DRAM-only mode after a license has expired.
- Failure of the *M3C* web server does not affect running applications but makes it difficult to manage and monitor them.
- And so on.

The Management Service is a single point of failure. A Management Service failure can be the result of the management host crashing; a network outage that cuts the Management Service off from data host agents; or another cause.

Since CPS serves the web UI, a CPS outage disables *M3C*.

There is no immediate effect on data host MVMASs if the Management Service fails. MVMASs continue to allocate PMEM, and TMS continues to manage PMEM and DRAM tier, on each individual data host.

If the failure is recoverable then the crashed Management Service restarts; in this case normally no interruption of the Data Service is noticed.

If Management Service remains offline, MVMAS on each data host misses the license management heartbeat and reverts to DRAM-only mode after a grace period of seven days.

Memory Machine Agent Failure

Failure of Memory Machine Agent on a data host cuts off that host from the Management Service. Memory Machine runs as normal on the host until the Memory Machine Agent is restarted by the host service configuration.

If for some reason the Memory Machine Agent remains unavailable, the MVMAS on each affected data host reverts to DRAM-only mode after a grace period of seven days.

Out of Memory Conditions

There are two cases of out-of-memory:

- An application instance can consume its entire allotted DRAM tier
- Application instances can fill up all available PMEM on a host

An application with insufficient DRAM tier can suffer severe performance issues. The *Memory Machine User's Guide* describes how to allocate DRAM tier to balance application performance against over-allocating DRAM capacity.

If application instances use up a host's PMEM, the result can be application slowdowns or failures just as it would for a DRAM-only system.

License Expiration

The Memory Machine Management Service generates alerts before, during, and after a license expiration. After a grace period, a host running on an expired license reverts to DRAM-only mode. This means that application instances can still read from PMEM, but can only write to DRAM, and that DRAM tiering is disabled.

Web UI Failure

Failure of CPS on the management host renders *M3C* unavailable.

Alert and Reporting Failures

Telemetry failure: If data telemetry ("Call Home") has been enabled for purposes of licensed usage reporting, then prolonged failure of telemetry (because of a prolonged disconnection from the internet, for example) eventually causes CPS to put Data Services into an unlicensed state. See License Expiration *(on page 11)*.

Email system failure: If the configured email system becomes unavailable, alerts cannot be delivered via email. They are still logged and available in *M3C*.

Logging failure: If the log directory becomes unavailable, logging stops. This would be unusual; the log directory is created by the installer with the correct permissions.

Database Failures

This database is a potential critical point of failure, especially for the Management Service.

MemVerge offers integration with a growing number of databases that provide varying levels of failover security. See Choosing a Database in the Memory Machine Help Center for details.

Hardware Failure Modes

This section describes the effects of hardware failure on Memory Machine, including host outages, network outages, and PMEM failure.

Data Host

A data host outage shuts down or crashes application instances on the host.

A data host can be restarted automatically or manually. By default when the Management Service is installed, host services are configured to restart the Management Service (CPS and Memory Machine Agents). Data services (MVMAS and Snapshot) can (and usually should) be configured to restart automatically as well, but this is not the installation default.

Application instances can be restored from persisted snapshots once MVMAS is running.

Management Host Failure

A short management host outage is equivalent to a Management Service outage. The crashed management host can be rebooted and the Management Service is restored with no interruption in memory management services on the data hosts.

If the management host becomes unavailable past seven days, then the MVMASs on the data hosts detect a license failure and revert to DRAM-only mode.

Network Outages

Besides affecting any application instances that require network access, network outages can affect Memory Machine resiliency primarily by cutting off communication between the Management Service and Memory Machine Agents on data hosts. See Management Host Failure *(on page 12)* above.

PMEM failures

Hardware failure of any memory component is rare. This is true of Intel Optane persisent memory as well. See for example the Intel publication Reliability, Availability, and Serviceability (RAS).